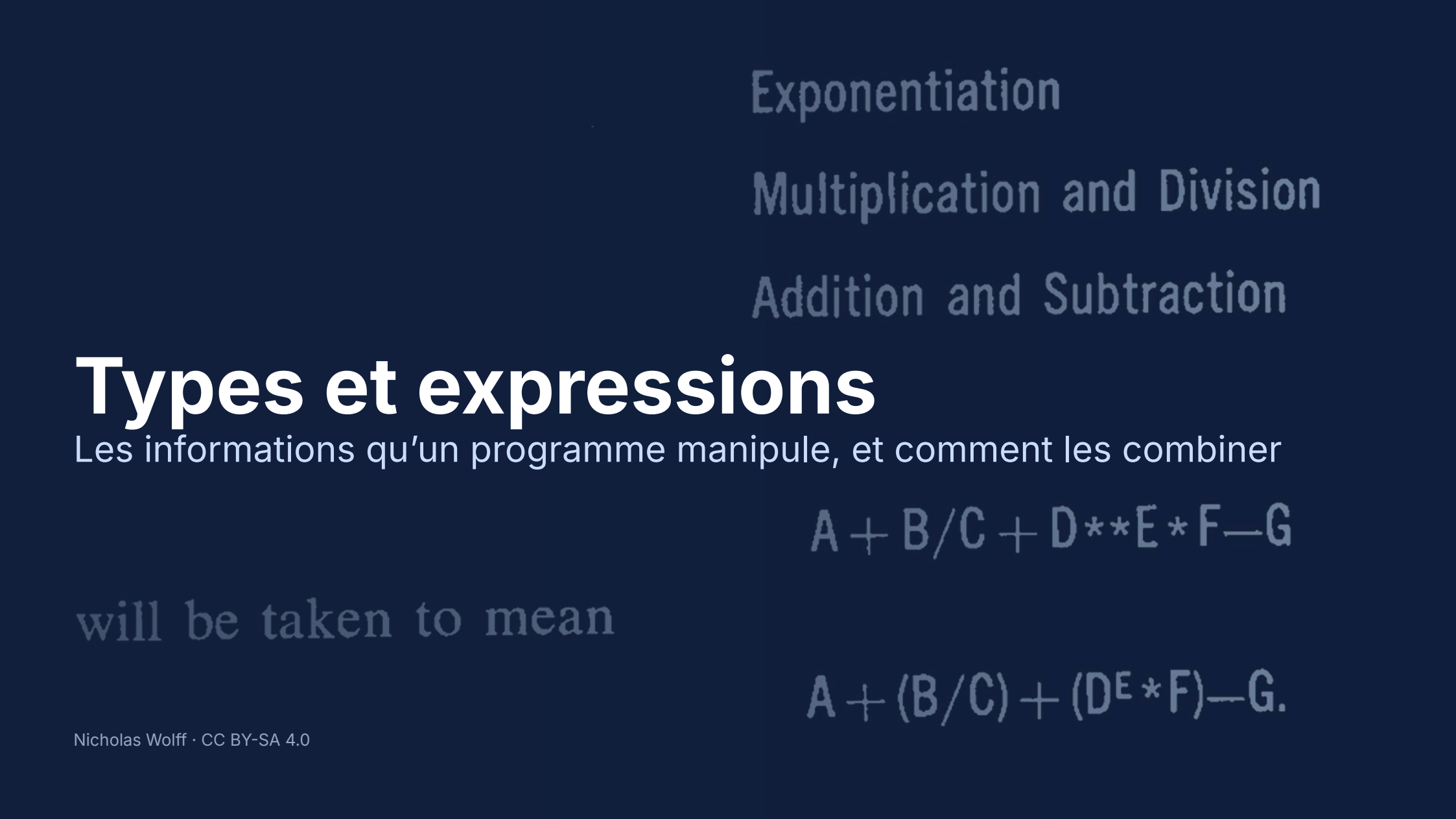


Types et expressions

Les informations qu’un programme manipule, et comment les combiner

Nicholas Wolff · CC BY-SA 4.0

Sommaire

1

Types, opérations, conversions

Les informations d’un programme ne sont pas toutes de même nature : quatre types, leurs opérations, les conversions

2

Expressions

Combiner des valeurs à l’aide d’opérateurs : évaluation, priorité, comparaisons

Exemples de données

Activité · 3 min

Lister toutes les natures d'informations différentes qu'on trouve dans ces deux tableaux et, pour chacune, les lignes concernées.

Club de fitness

Prénom	Marc	Sophie
Âge	34	22
Taille (m)	1.82	1.65
Compte Premium	Oui	Non
Activités préférées	Yoga, Surf	Tennis, Boxe

Magasin informatique

Référence	LAP-PRO	MOU-MINI
Catégorie	Ordinateur	Accessoire
Quantité	12	45
Prix (CHF)	899.95	24.50
En promotion	Non	Oui
Ventes (3 derniers mois)	5, 8, 3	15, 22, 18

Exemples de données

Activité

Lister toutes les natures d'informations différentes qu'on trouve dans ces deux tableaux et, pour chacune, les lignes concernées.

Nombres entiers	Âge, Quantité
Nombres décimaux	Taille (m), Prix (CHF)
Texte	Prénom, Référence, Catégorie
Oui ou non	Compte Premium, En promotion
Liste de valeurs	Activités préférées, Ventes (3 derniers mois)

Valeurs et types

`34`, `"Marc"`, `1.82` et `True` sont des *valeurs*.
Chaque valeur a un *type*. Python a quatre types de base :

`int` : les **nombres entiers**.

```
34    0    1    -235098
```

`float` : les **nombres décimaux**, écrits avec un point.

```
1.82    33.33333    0.0    .25    2.0    3.1415
```

`str` : les **chaînes de caractères**, du texte délimité par des `"`.

```
"Marc"    "bande dessinée"    "1"    "True"      " "       ""
                                                un espace   vide
```

`bool` : les **booléens**, qui n'ont que deux valeurs possibles.

```
True     False
 oui      non
```

Opérations sur les nombres (`int`, `float`)

Opérations simples. Si l'un des nombres est décimal, le résultat est un nombre décimal.

```
3 - 2  → 1
```

```
1.5 * 2  → 3.0
```

```
2 ** 3  → 8
```

puissance

Division. Le résultat est toujours un nombre décimal.

```
1 / 2  → 0.5
```

```
-4 / 2  → -2.0
```

Division entière et modulo. Pour des nombres entiers, les résultats sont des nombres entiers.

Division entière (le quotient)

```
7 // 3  → 2
```

Modulo (le reste)

```
7 % 3  → 1
```

(7 = 2 × 3 + 1)

Comme en maths, la multiplication se fait avant l'addition. Si on veut modifier l'ordre, on ajoute des parenthèses.

```
1 + 2 * 3  → 7
```

```
(1 + 2) * 3  → 9
```

Opérations sur les chaînes de caractères (`str`)

Entre des chaînes de caractères, + est une *concaténation* : il colle les chaînes bout à bout.

```
"porte" + "manteau" → "portemanteau"
"1" + "2" → "12"
"bob" + "" → "bob"
```

On ne peut pas concaténer une chaîne avec un nombre ou un booléen :

```
"bob" + 1 → Erreur
"1" + 1 → Erreur
"Vrai" + True → Erreur
```

Opérations sur les booléens (`bool`)

On peut effectuer des opérations logiques :

Négation :

```
not True  → False
not False → True
```

le contraire

Et :

```
True and True  → True
True and False → False
False and False → False
```

`True` seulement si les deux sont `True`

Ou :

```
True or True  → True
True or False → True
False or False → False
```

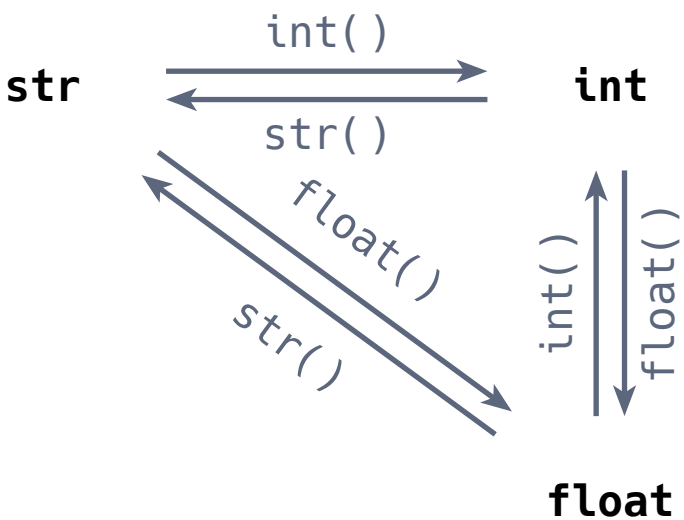
`True` si au moins un des deux est `True`

Conversions entre types

Python fait une distinction forte entre les types : 4 et `"4"` ne sont pas la même valeur. Cela permet d'éviter de faire des erreurs en programmant.

Il est cependant possible de demander à Python de *convertir* une valeur d'un type à l'autre. Pour convertir vers un type, on utilise la fonction qui porte son nom : `int()`, `float()`, `str()`.

Les booléens ne se convertissent pas de la même manière : on les laisse de côté pour l'instant.



Conversions : exemples

Entre chaîne et nombre :

```
str(4) → "4"
str(4.5) → "4.5"
```

```
int("4") → 4
float("4.5") → 4.5
```

Entre nombres :

```
float(4) → 4.0
int(4.9) → 4    (troncature : on enlève la partie décimale)
```

Il n'est pas toujours possible de convertir :

```
int("4.5") → Erreur    (la chaîne ne contient pas un nombre entier)
int("Bob") → Erreur
```

Sommaire

1

Types, opérations, conversions

Les informations d’un programme ne sont pas toutes de même nature : quatre types, leurs opérations, les conversions

2

Expressions

Combiner des valeurs à l’aide d’opérateurs : évaluation, priorité, comparaisons

Expressions

Une *expression* combine des *valeurs* à l'aide d'*opérateurs*.

On *évalue* une *expression* pour trouver sa *valeur*.

```
100 * 99  →  9900
```

L'évaluation d'une expression qui contient plusieurs opérateurs se fait en plusieurs étapes.

```
2 + 3 + 4  →  5 + 4  →  9
```

Comme en maths, la priorité des opérateurs s'applique.
Les parenthèses permettent de décider de l'ordre.

```
1 - 2 * 3    →  1 - 6   →  -5
(1 - 2) * 3  →  -1 * 3  →  -3
```

Opérateurs de comparaison en Python

En français	En maths	En Python
Supérieur	>	>
Inférieur	<	<
Supérieur ou égal	≥	>=
Inférieur ou égal	≤	<=
Égal	=	== (deux signes =)
Différent	≠	!=

Comparaisons

Le résultat d'une comparaison est toujours un booléen.

```
1 > 2                  →  False
1 <= 2                 →  True
1 > 2 or 1 <= 2        →  False or True    →  True
1 > 2 and 1 <= 2       →  False and True   →  False
"Babar" > "Alexia"     →  True    (ordre alphabétique)
```

Priorité : les comparaisons d'abord, puis `not`, puis `and`, puis `or`.

Sources

Contenu

Python — Priorité des opérateurs
docs.python.org

Python — Comparaisons
docs.python.org

Images

IBM – la priorité des opérateurs dans le premier manuel de FORTRAN (1956, domaine public)
bitsavers.org